

Universidade Federal da Fronteira Sul – UFFS
Curso: Ciência da Computação (2ª fase)
Disciplina: Circuitos Digitais
Professores: Geomar Andre Schreiner e Luciano Lores Caimi

Niumar Girardi

TRABALHO FINAL
Trabalho de Circuitos digitais: jogo de batalha naval

Links:
[Tinkercad](#)
[Video](#)

Chapecó
2025

Trabalho de Circuitos digitais: jogo de batalha naval

Descrição

Disciplina: Circuitos Digitais

Data limite de entrega: 13/07/2025

Grupo: 2 pessoas

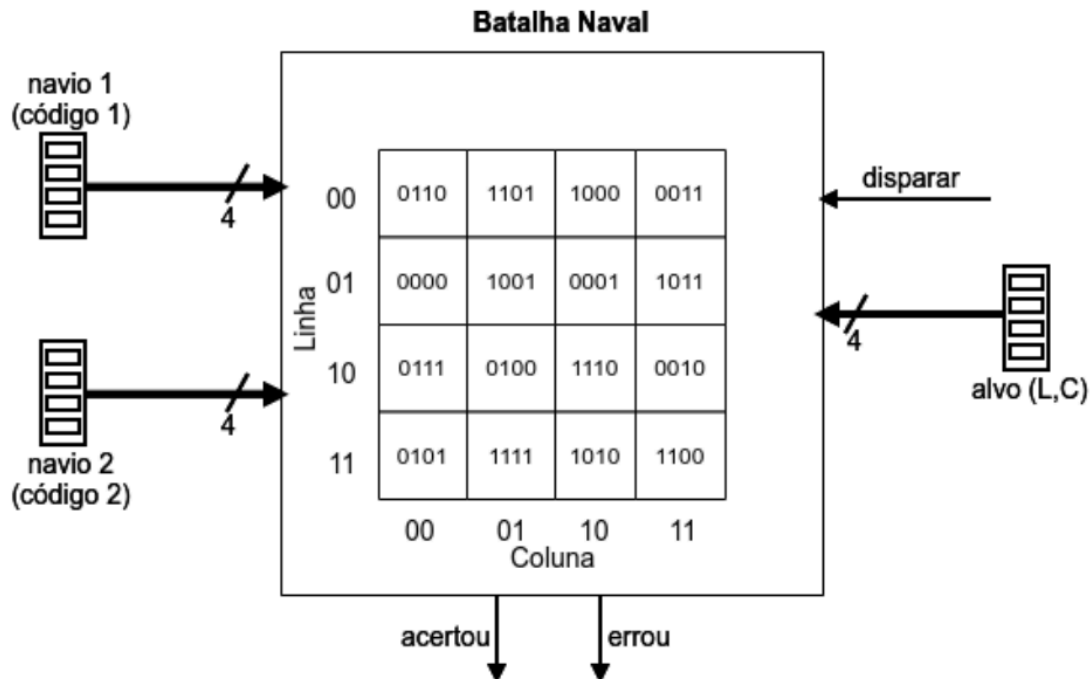
O que entregar: arquivo compactado cujo nome é "nome_1-nome_2" contendo:

- Relatório incluindo:
 - Apresentação
 - Descrição da solução:
 - Relato da estratégia aplicada
 - Codificação do campo
 - Tabela-verdade
 - Simplificações
 - Circuito completo usando portas lógicas utilizando o Logisim
 - Circuito completo usando CIs (74XX) usando o tinkercad (incluir link)
 - Conclusão
- Arquivo do projeto Logisim
- Arquivo do projeto Tinkercad
- Link para um vídeo mostrando as funcionalidades e a execução correta do jogo

O trabalho proposto simula o jogo de batalha naval. O "mar" é formado por uma matriz de 4x4 posições e o dono deste mar territorial deve colocar 2 navios no mesmo. Para posicionar os navios o dono utiliza 2 conjuntos com 4 chaves onde, de forma codificada, é informada a posição dos navios no mar (cada posição da matriz possui um código associado).

O adversário terá um conjunto de 4 chaves à sua disposição para informar a posição do alvo e uma entrada para solicitar disparo em direção aos navios. As chaves do adversário não possuem codificação associada e informam diretamente a posição (linha, coluna) onde o disparo é realizado.

Por exemplo, as chaves de posição do alvo informando "1011" indicam o disparo na linha "10", coluna "11". Na figura abaixo a posição da linha "1011" está codificada com o valor "0010". Desta forma, se algum dos navios (entradas "navio_1" ou "navio_2") possuir o código "0010" a saída acertou deve ir para nível lógico 1 quando a entrada "disparar" for pressionada pelo adversário. Quando o disparo errar o alvo a saída "errou" deve ir para nível lógico 1.



De forma resumida, uma solução consiste em:

- 1) Transformar um código em outro (do código da posição dos navios em posição da linha e coluna onde o código se encontra);
- 2) Comparar a posição decodificada dos navios (linha, coluna), com o valor de linha e coluna do alvo informado pelo adversário.

Para a montagem no ThinkerCad as saídas acerto e erro são LEDs, e os códigos das posições dos navios devem utilizar "interruptores". Use mais de um protoboard para facilitar a explicação, utilize também recursos como as cores das ligações.

Não será necessária a apresentação do trabalho, apenas o vídeo já será considerada como apresentação. Porém, por amostragem, alguns alunos serão convocados para a apresentação do trabalho.

Cada grupo deve indicar a codificação para a matriz que utilizará conforme o arquivo do link a seguir:

https://docs.google.com/document/d/1hEjW_V4x9VtqwzR4OOBPiFqxpwlh3aO4y0zPfYzPM0/edit?usp=sharing

Codificação Escolhida

Com base no documento passado pelo professor escolhido a codificação de Número 6:

Codificação 6: Niumar Girardi

1010	0100	0111	1101
0010	1000	1110	0101
1011	0000	1111	0110
1001	0001	1100	0011

Solução:

Para realizar a solução e gerar o circuito alguns passos devem ser realizados antes de iniciarmos a construção do jogo em si.

Análise do que deve ser feito:

Entendido que o tabuleiro/matriz se trata do mar do Jogador 1 onde o mesmo deve posicionar os seus 2 barcos e após o jogador 2 deve tentar adivinhar a posição para ganhar o jogo;

Cada posição é um número de 4 bits o que significa que para o jogador 1 posicionar "Um" barco o mesmo deve dar um código de 4 bits, então ele posiciona o barco 1 usando uma entrada de 4 bits e posiciona o barco 2 usando mais uma entrada de 4 bits, ou seja o Circuito deve ter 2 entradas para os barcos cada uma contendo o código de 4 bits (A, B, C e D)

Após posicioná-los os barcos o jogador 2 usará uma 3ª entrada de 4 bits para tentar adivinhar a posição dos barcos do jogador um e irá receber uma saída de 0 ou 1 para saber se acertou.

Até agora 2 entradas de 4 bits (barcos), 1 de 4 bits (tiro), 1 de 1 bit (botão pra tiro) e uma saída de 1 bit.

Quanto ao Tabuleiro agora devemos pensar em como vamos posicionar os barcos, Inicialmente pensei em criar 2 codificadores um para cada barco mas isso acabaria usando "muitos" circuitos e iria ficar uma bagunça quando eu fosse passar para o tinkercad. então pensei em criar um codificador somente para o Tiro, exemplo o jogador 1 iria colocar direto o código de onde estariam os barcos Usando os valores do MAR, após o Jogador irá dar o tiro usando o valor de fora do mar vou chamar AB\CD:

Tabela para explicação Abaixo:

O jogador 1 usaria o Códigos em Azul(O mar) para posicionar o barco e o Jogador 2 usaria o em amarelo(AB\CD) para dar o tiro.

AB\CD	00	01	10	11
00	1010	0100	0111	1101
01	0010	1000	1110	0101
10	1011	0000	1111	0110
11	1001	0001	1100	0011

Digamos que o jogador 1 escolhe as seguintes “Casas”

AB\CD	00	01	10	11
00	1010	0100	0111	1101
01	0010	1000	1110	0101
10	1011	0000	1111	0110
11	1001	0001	1100	0011

O jogador 2 teria que usar os valores em amarelo para tentar acertar, se ele colocasse 1010 ele erraria pois seria o valor 1111, agora se ele colocasse o valor 0000, acertaria pois acharia o 1010

Então basta criarmos um codificador para os tiros, pois a entradas dos barcos o jogador 1 já adiciona codificada.

Até agora: 2 entrada de 4bits(barcos), 1 de 4bits(tiro), 1 de 1bit(botão pra tiro), uma saída de 1bit, e 1 codificador para as entradas dos Tiros

Temos entradas e codificadores, sabemos o'que deve sair na saída, mas ainda nao vimos como vamos fazer isso, bom é bem simples, só fazer um Verificador\comparador. o comprador deve verificar se a as entradas são iguais, dá para usar uma xor negada que quantos valores por ex: no 1 bit, do barco e 0 e do tiro e 0, 0 xor 0 a saída é 0 mas se negarmos saída, 1 que indicará igualdade.

Ou seja pegariamos os 4 bits do barco e comparamos um por um com o os 4 bits do tiro

Agora temos todos os requisitos do circuito:

2 entrada de 4bits(barcos), 1 de 4bits(tiro), 1 de 1bit(botão pra tiro), uma saída de 1bit, e 1 codificador para as entradas dos Tiros e 2 comparadores(um para cada barco)

Construção da Lógica dos Codificadores:

Para construir o codificador vamos ter que saber quais portas vamos usar, usamos karnaugh para já termos o circuito simplificado mas para isso antes deve ser feito a tabela verdade para sabermos as suas saídas

Tabela Verdade:

Uma tabela verdade de 4 entradas ($2^4=16$ linhas) com 4 saídas: Sa, Sb, Sc e Sd, ou Bit3, bit2, bit1 e bit0.

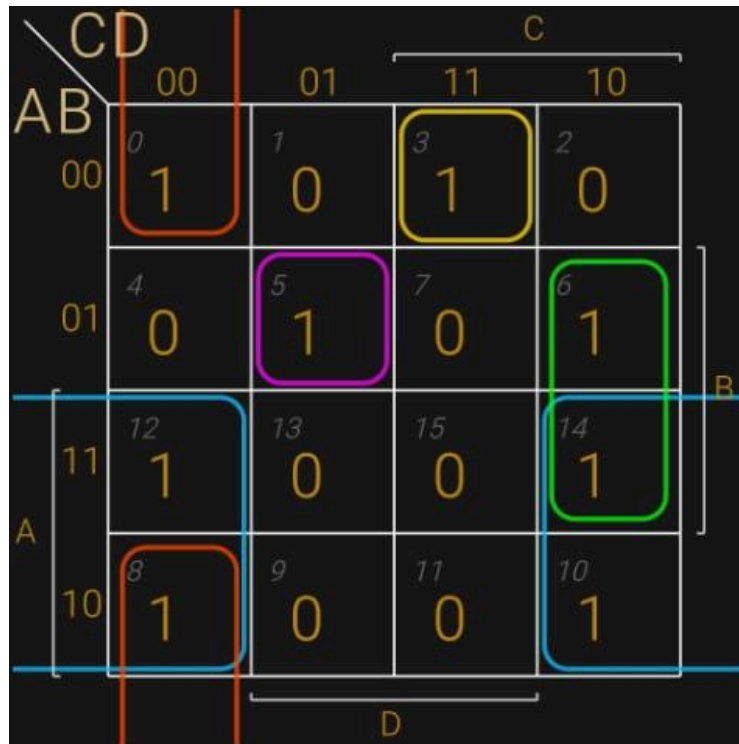
AB\CD	00	01	10	11
00	1010	0100	0111	1101
01	0010	1000	1110	0101
10	1011	0000	1111	0110
11	1001	0001	1100	0011

A	B	C	D	Sa	Sb	Sc	Sd
0	0	0	0	1	0	1	0
0	0	0	1	0	1	0	0
0	0	1	0	0	1	1	1
0	0	1	1	1	1	0	1
0	1	0	0	0	0	1	0
0	1	0	1	1	0	0	0
0	1	1	0	1	1	1	0
0	1	1	1	0	1	0	1
1	0	0	0	1	0	1	1
1	0	0	1	0	0	0	0
1	0	1	0	1	1	1	1
1	0	1	1	0	1	1	0
1	1	0	0	1	0	0	1
1	1	0	1	0	0	0	1
1	1	1	0	1	1	0	0
1	1	1	1	0	0	1	1

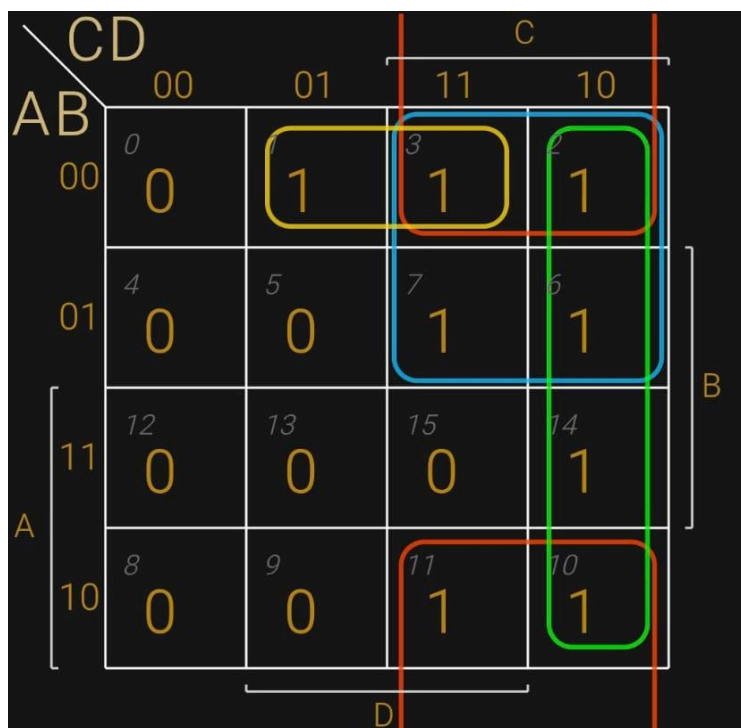
Mapa de Karnaugh e Circuito simplificado:

Usado o Aplicativo Karnaugh map para melhor visualização do mapa.

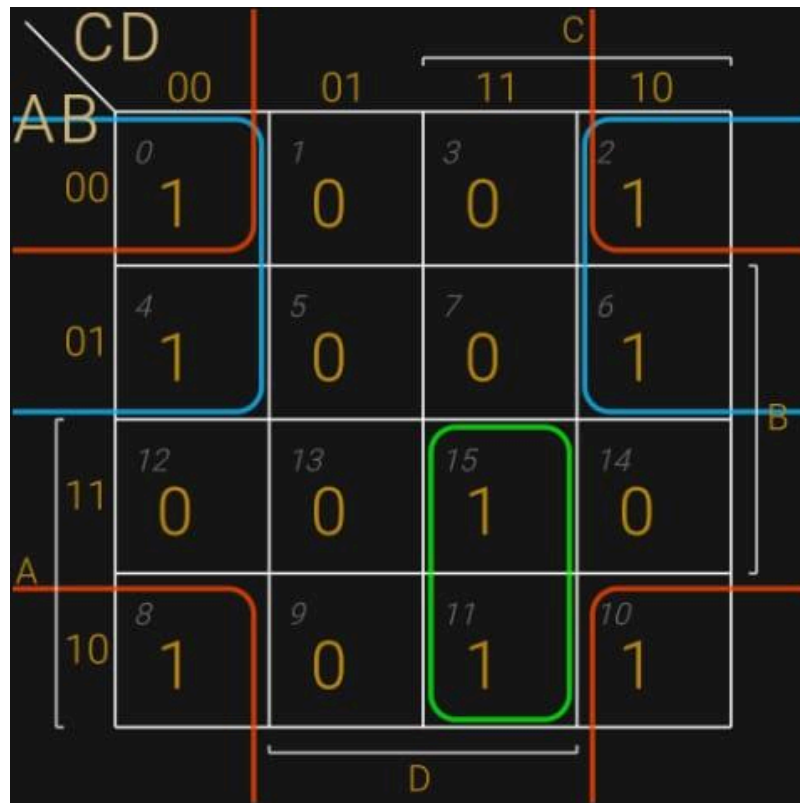
Sa: $AD! + B!C!D! + BCD! + A!B!CD + A!BC!D$



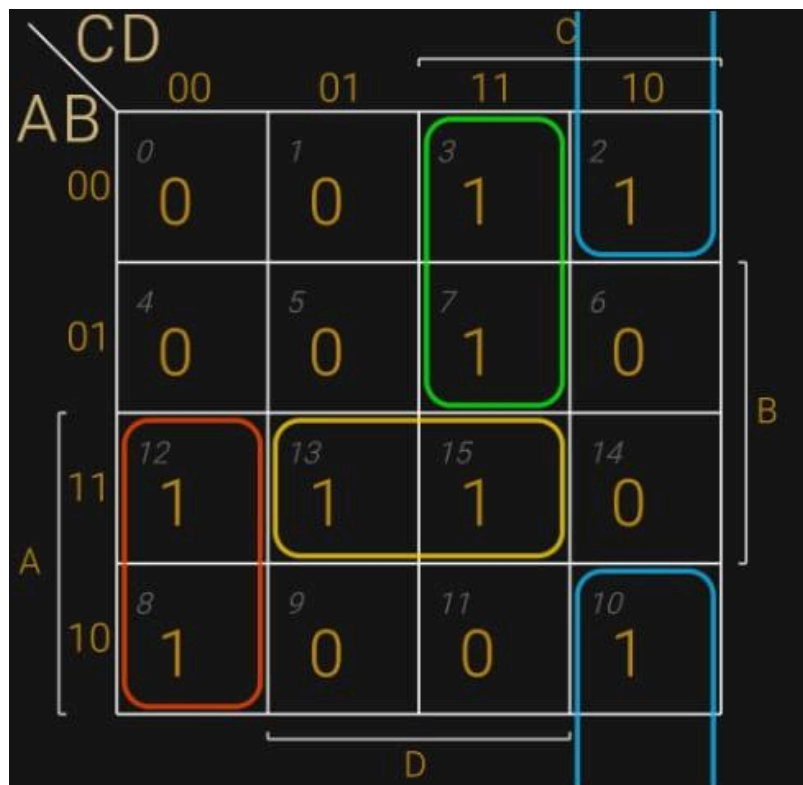
Sb: $A!C + B!C + CD! + A!B!D$



Sc: $A!D! + B!D! + ACD$



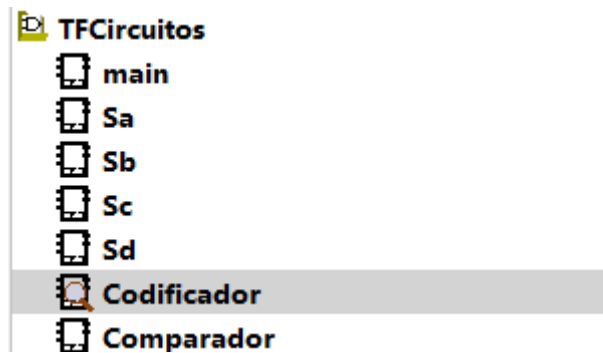
Sd: $B!CD! + AC!D! + A!CD + ABD$



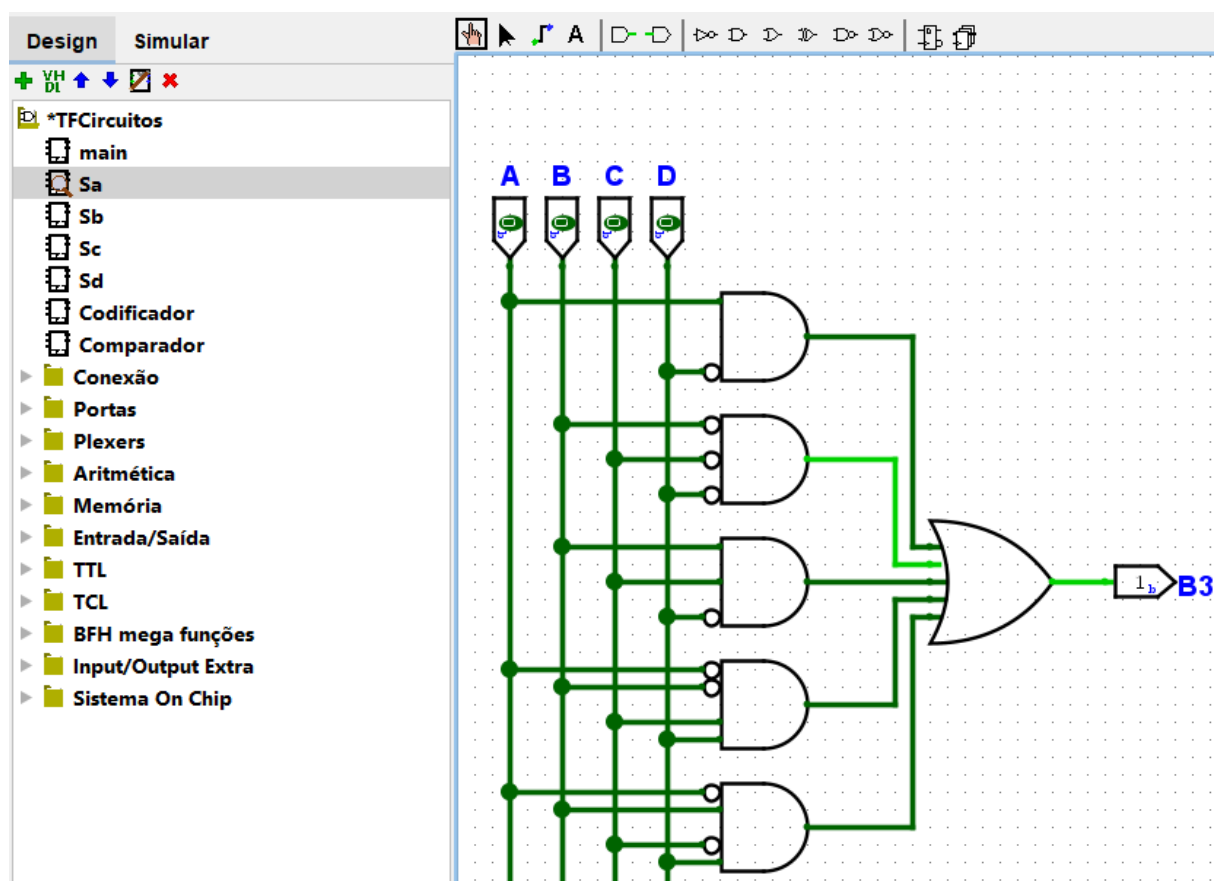
Logisim:

Agora que temos a simplificação da expressão booleana e temos a tabela verdade, basta aplicar a lógica no logisim

No logismo teremos 7 “Funções”: main, Sa , Sb, Sc, Sd, codificador e comparador.

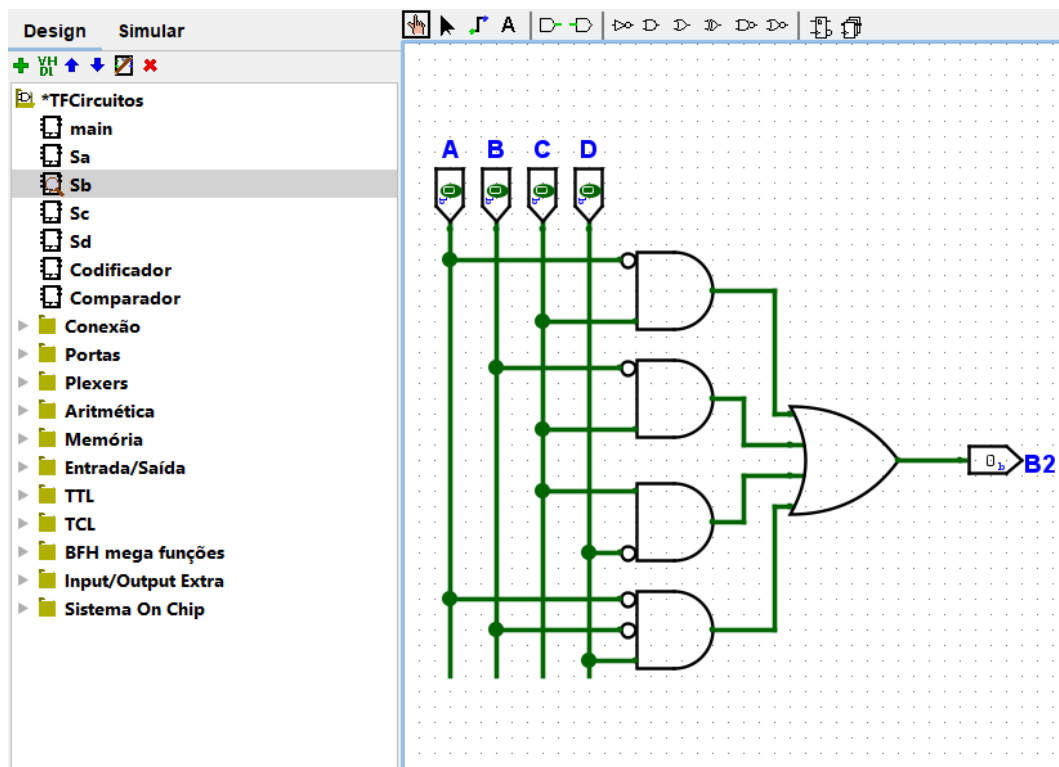


Sa: $AD! + B!C!D! + BCD! + A!B!CD + A!BC!D$



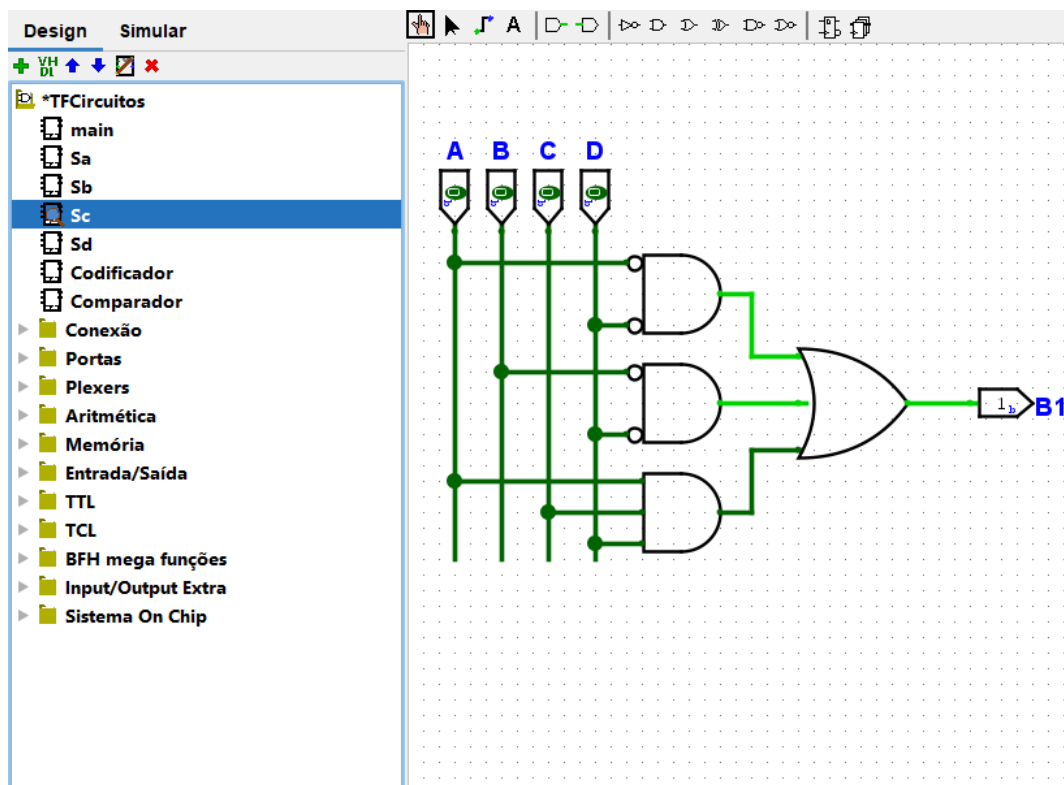
Esse irá retornar a “última casa” o Bit3 ex: 0000

Sb: $A!C + B!C + CD! + A!B!D$



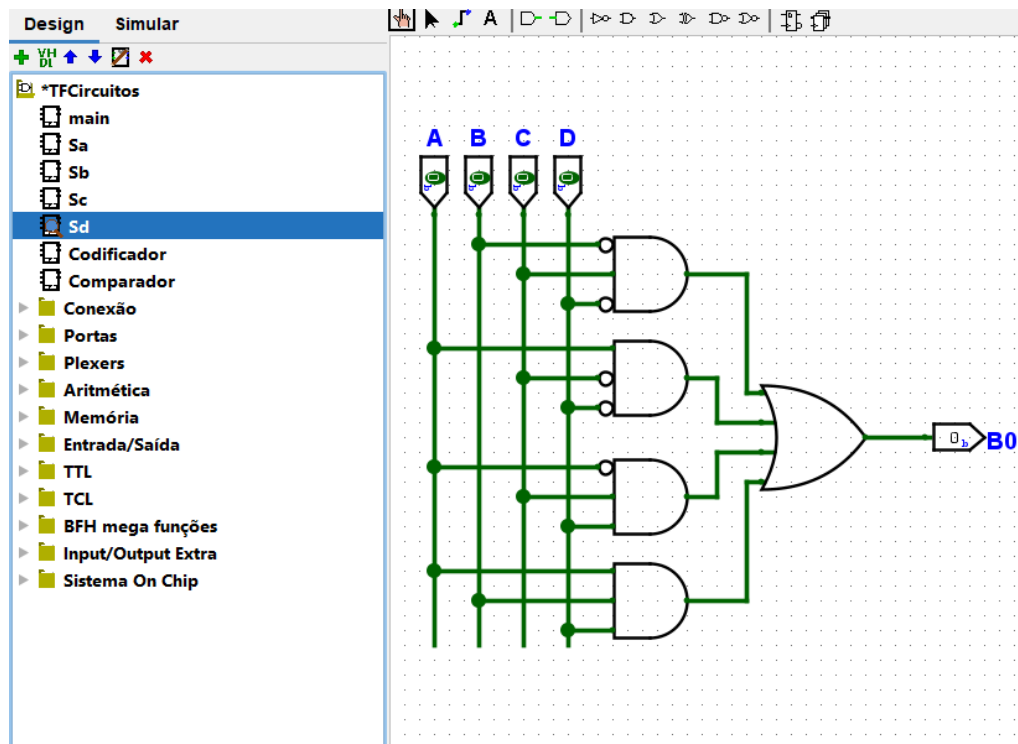
Esse irá retornar a “penúltima casa” o Bit2 ex: 00**0**0

Sc: $A!D! + B!D! + ACD$



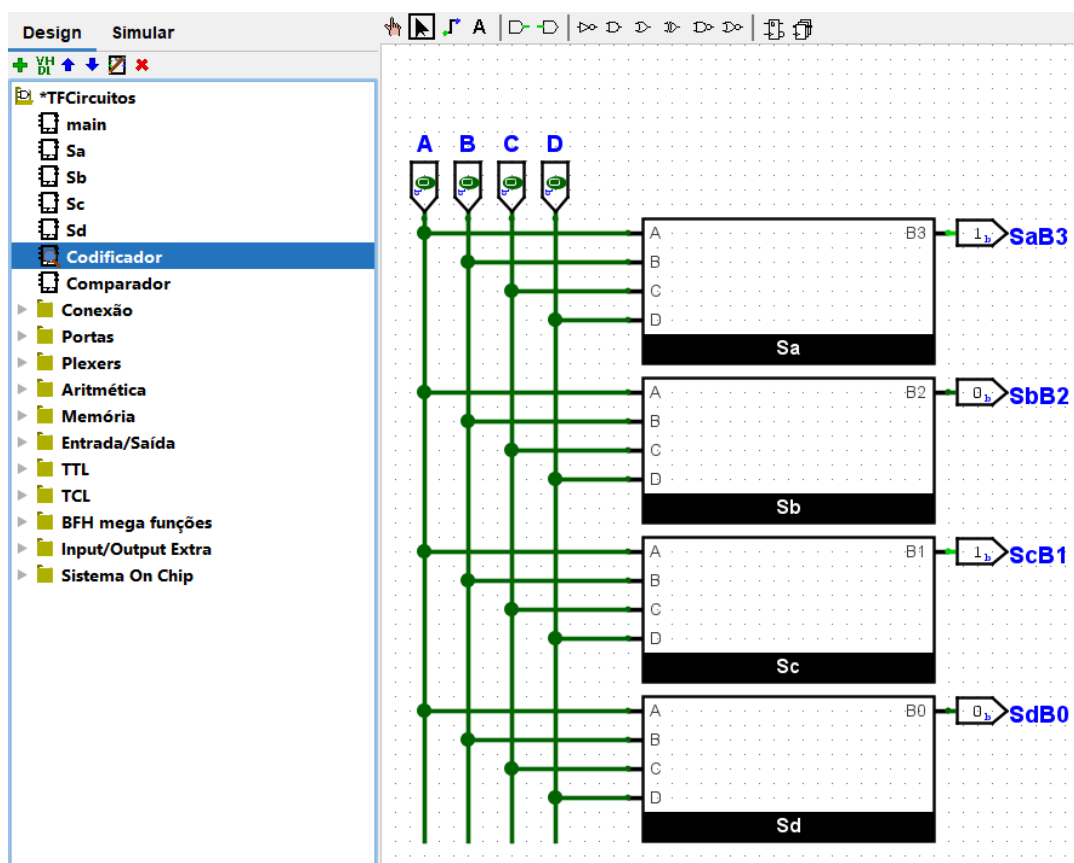
Esse irá retornar a “segunda casa” o Bit1 ex: 00**0**0

Sd: B!CD! + AC!D! + A!CD + ABD



Esse irá retornar a “primeira casa” o Bit0 ex: 000**0**

Codificador:



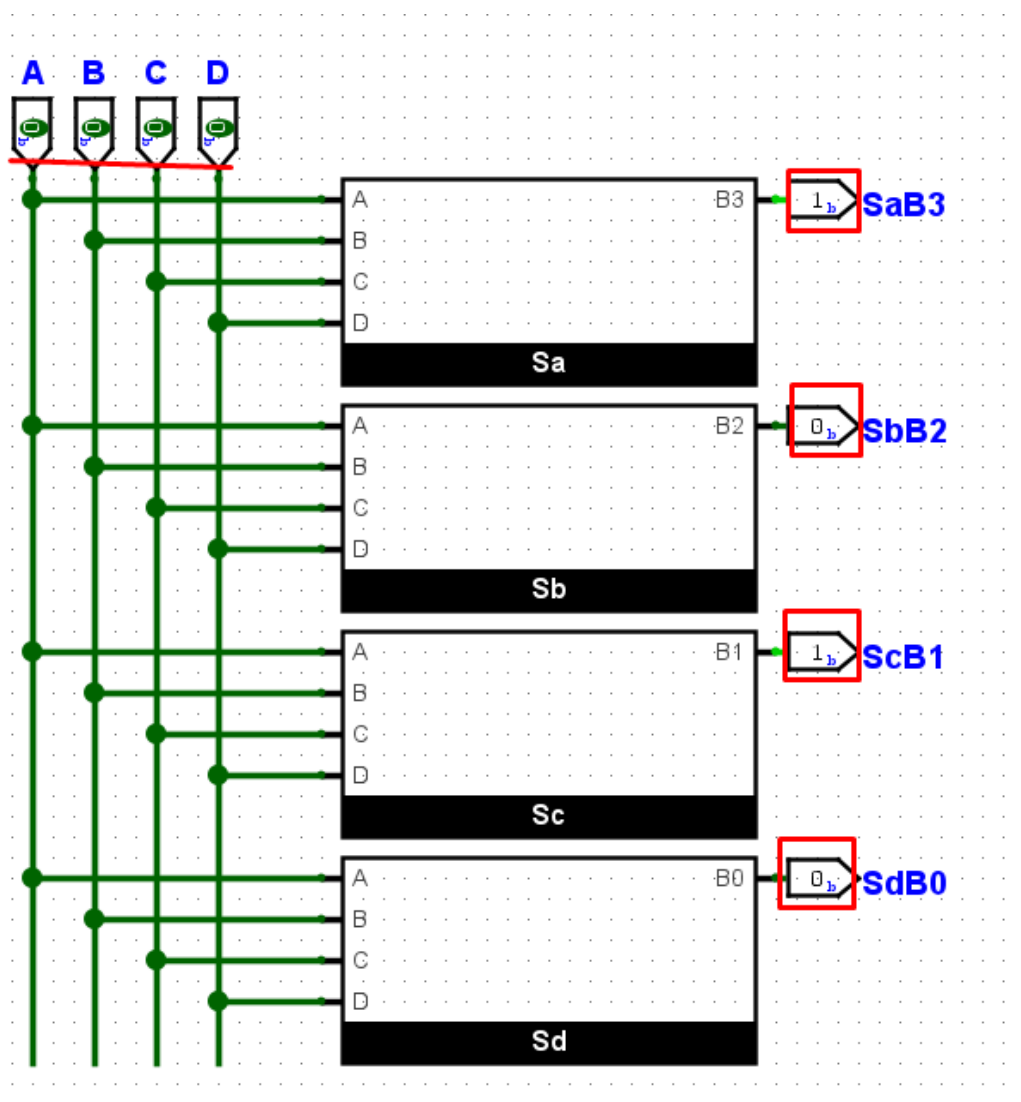
O codificador é bem simples irá pagar uma entrada de 4 bits e retorna uma saída de 4 bits

Ex se entrarmos com 0000

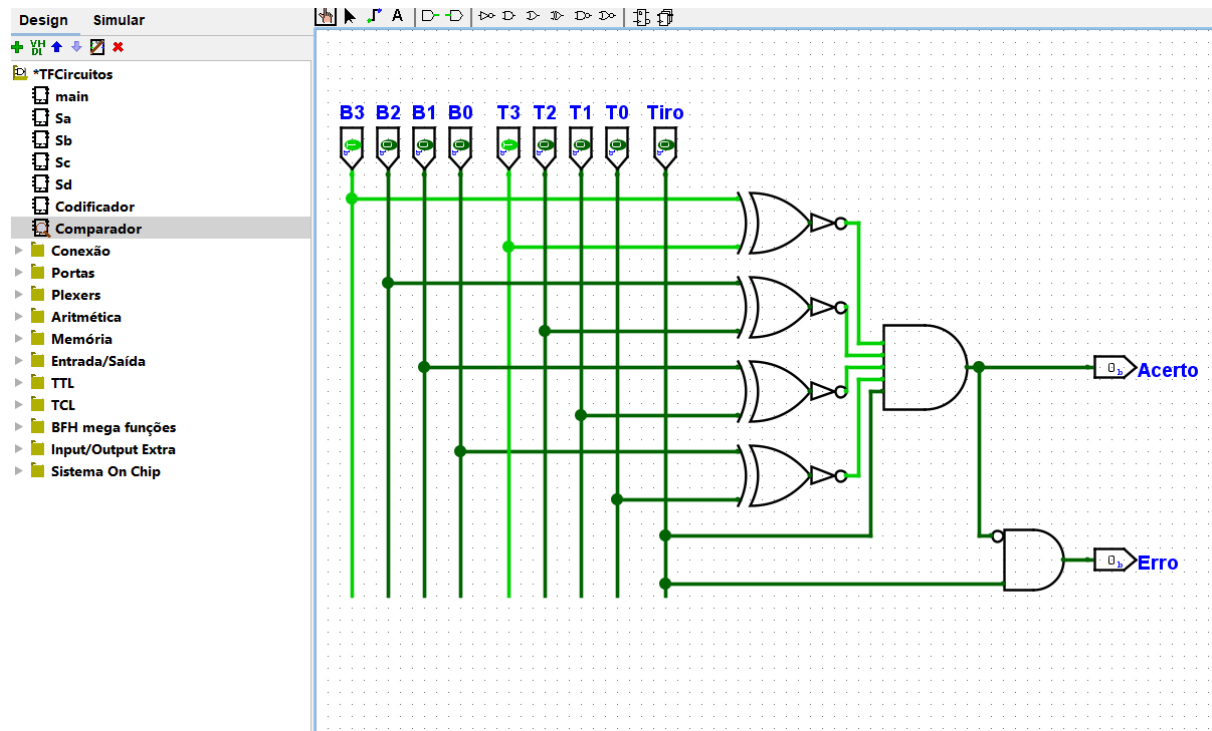
Ele utiliza os circuitos anteriores da mesma maneira que foi aplicado no trabalho do display de sete segmentos

AB\CD	00	01	10	11
00	1010	0100	0111	1101
01	0010	1000	1110	0101
10	1011	0000	1111	0110
11	1001	0001	1100	0011

Ele nos devolverá 1010



Comparador:

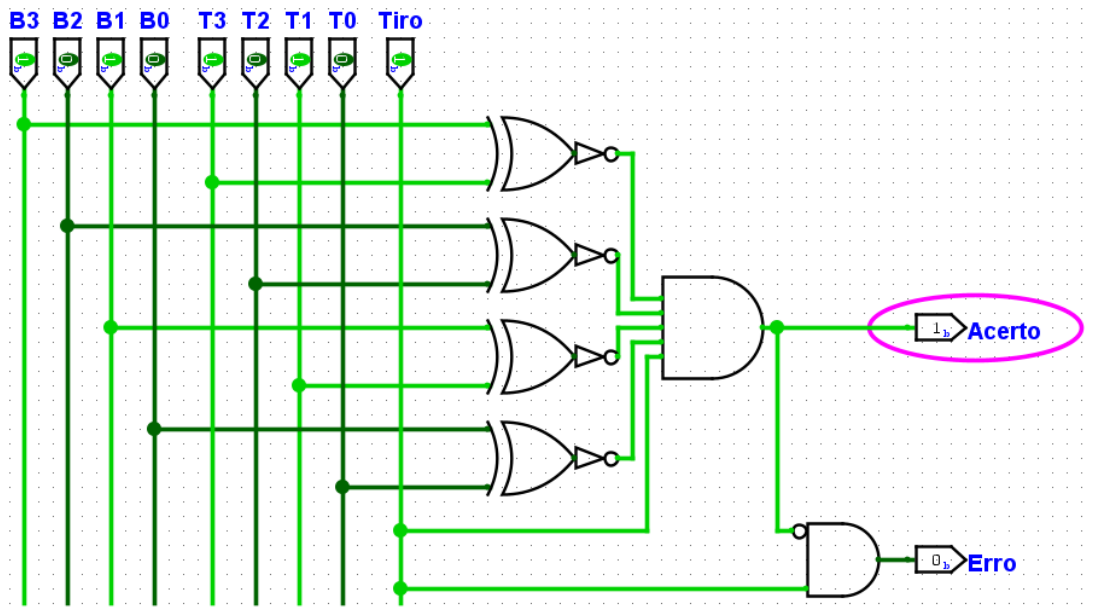


O comparador recebe duas entradas de de 4 bits e irá comparar bit a bit se elas são iguais

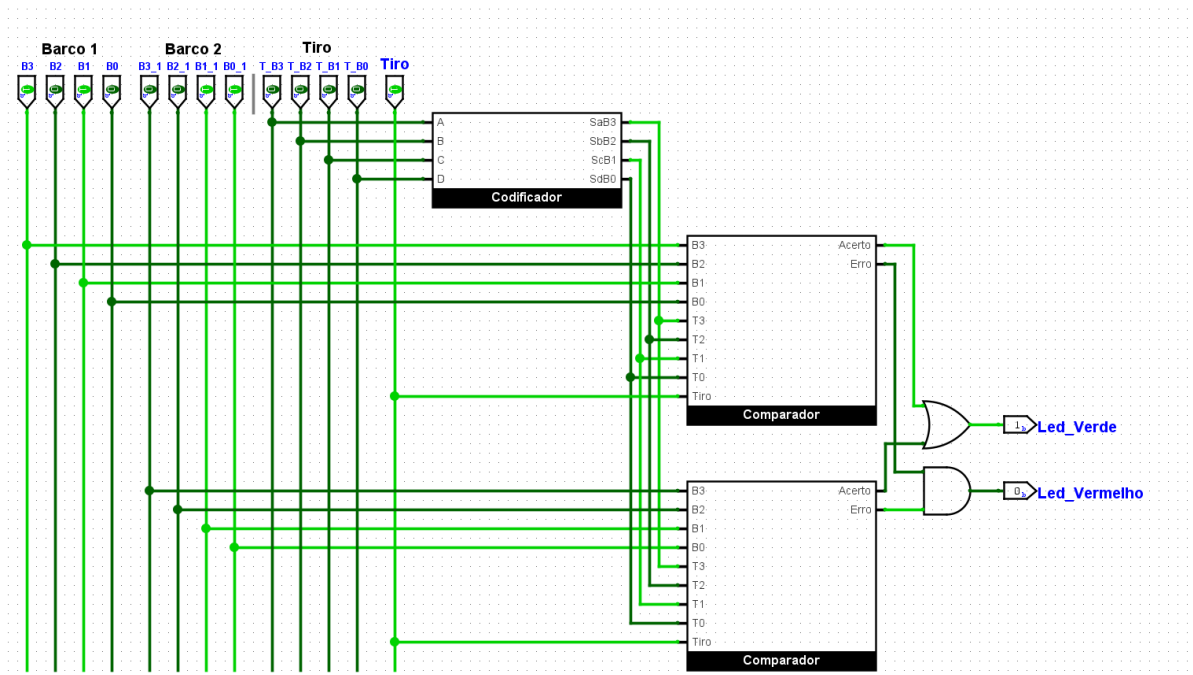
Se observar na imagem acima vai ver que ele usa de portas xnor para testar, também vai perceber o caso do bit a bit, B3 está conectado a T3.

A lógica é bem simples. Sabemos que o operador XOR retorna 1 somente quando os dois valores de entrada são diferentes. Se o negarmos, estaremos retornando 1 para valores iguais, que é exatamente o que queremos verificar no caso bit a bit.

Após informar as entradas B (referindo-se ao barco) e T (a posição de onde vamos dar o tiro), apertamos o botão "Tiro" para saber se acertamos ou não.



main:



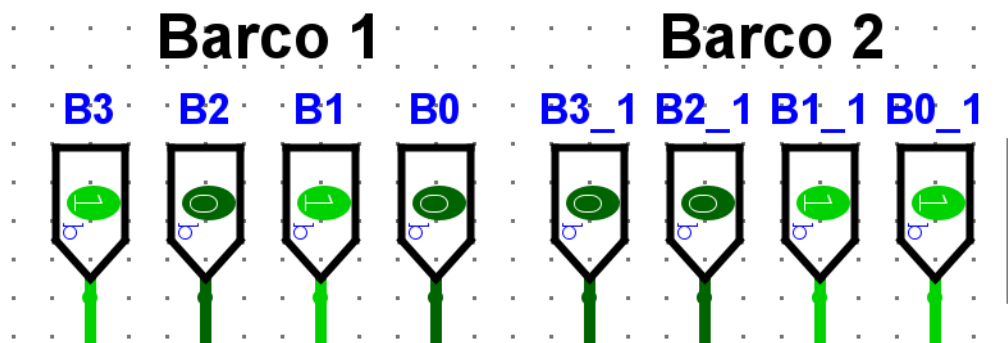
O main irá juntar toda a nossa lógica até o momento

Passo a passo:

Jogador 1 irá informar onde estão seus dois barcos

AB\CD	00	01	10	11
00	1010	0100	0111	1101
01	0010	1000	1110	0101
10	1011	0000	1111	0110
11	1001	0001	1100	0011

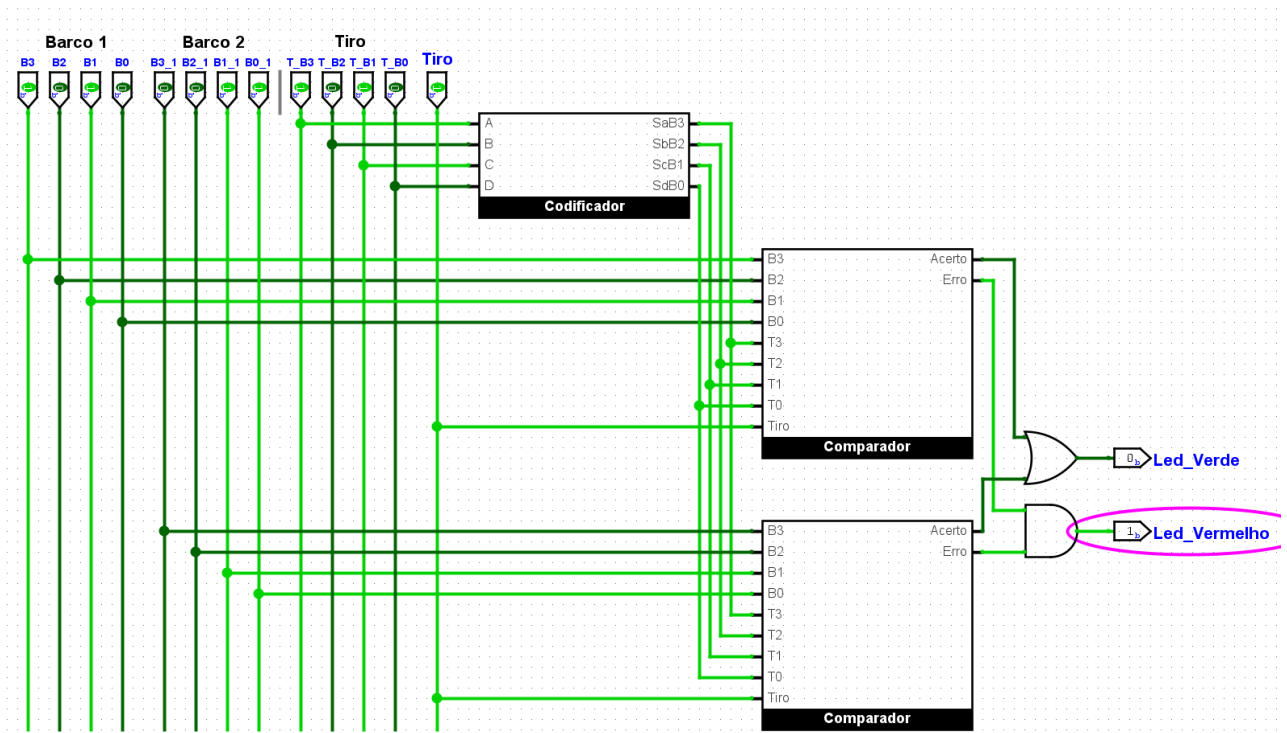
Ou seja irá passar as entradas 1010 para o barco 1 e 0011 para o barco 2



Após o Jogador 2 ira passar a entrada do Tiro ex: 1010

AB\CD	00	01	10	11
00	1010	0100	0111	1101
01	0010	1000	1110	0101
10	1011	0000	1111	0110
11	1001	0001	1100	0011

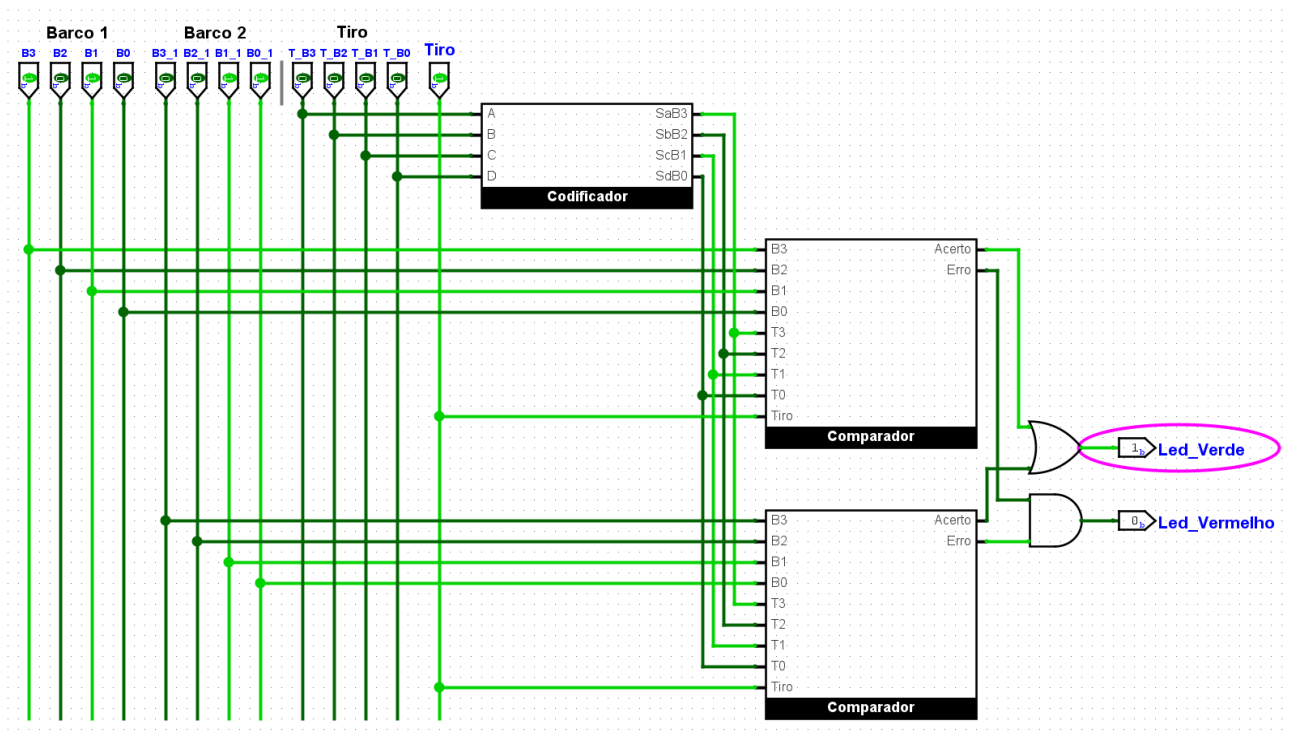
Após o Jogador passar a entrada 1010 ela ser codificada e sairá 1111 que entrara no comparador e irá ser verificada com algum barco 1111 é igual a 1010?, não então liga o led vermelho.



Agora ele tentará novamente mas com a entrada 0000:

Ela será codificada e sairá 1010, que será comparada ao barco 1010 e irá retornar led verde.

Imagem Na próxima página:



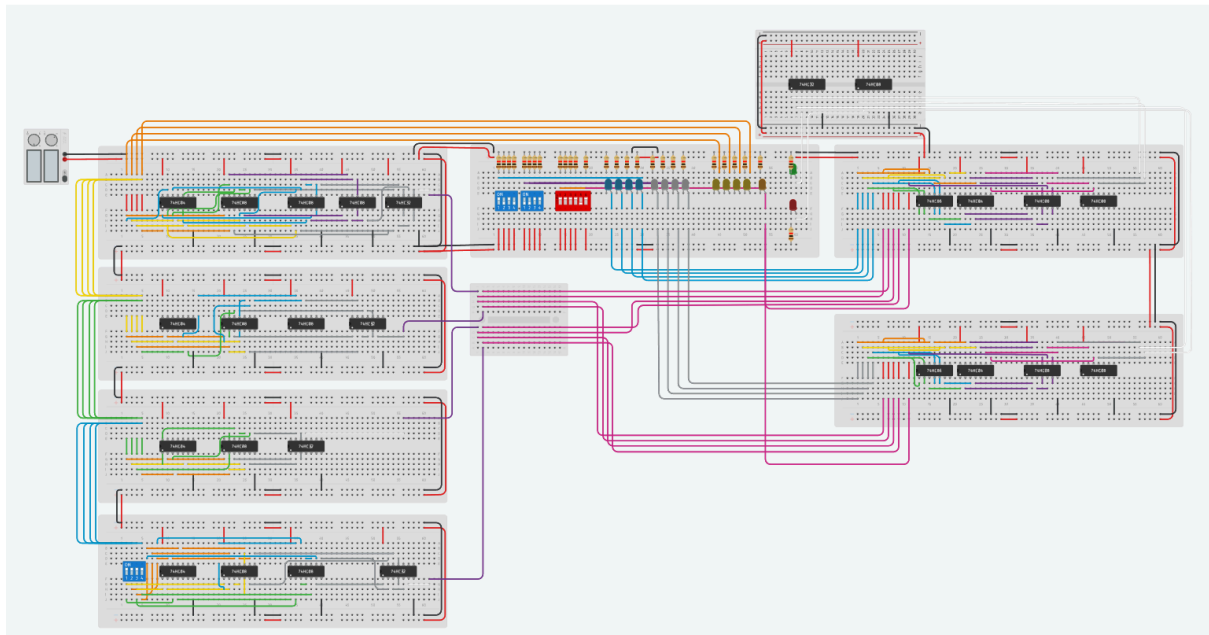
Tinkercad:

Link de Compartilhamento:

<https://www.tinkercad.com/things/htt69AOIU3F-tfcircuitos/editel?returnTo=https%3A%2F%2Fwww.tinkercad.com%2Fdashboard&sharecode=TXHFXS5vsmzXTTJmIKUiKYZ6BNVopoQAJCt9loeQOz8>

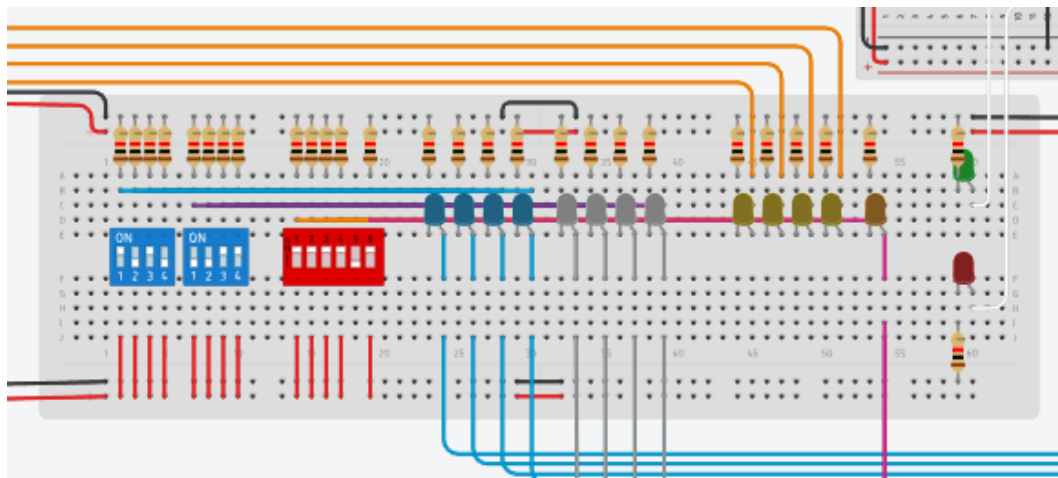
Abaixo estão as imagens do Tinkercad com a explicação da organização de como a lógica do circuito foi passada para a estrutura na bancada.

Circuito completo:



Estrutura do circuito por partes:

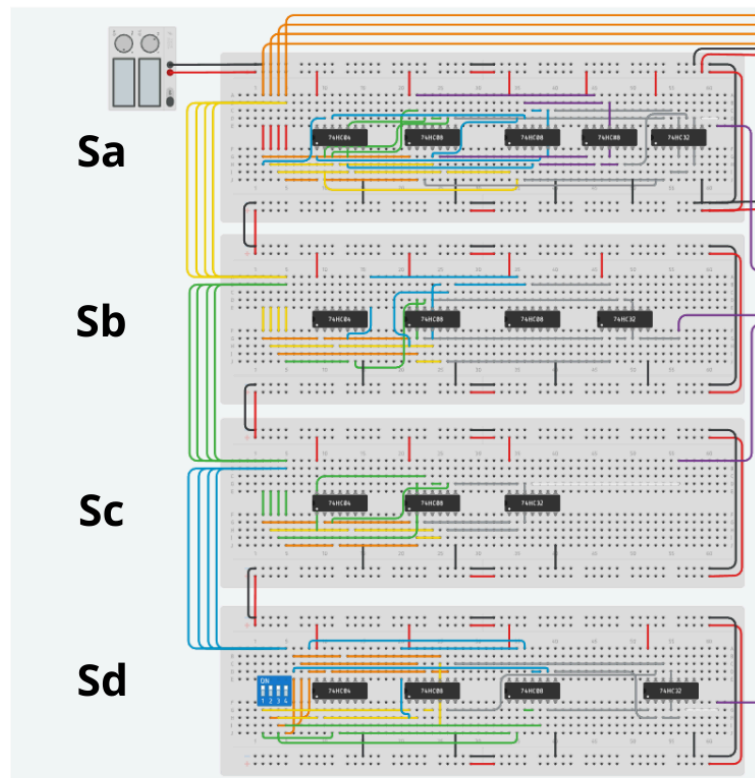
- Painel de interação do circuito



Nessa parte há dois tipos de interruptores DIP DPST x4 para os barcos e DIP DPST x6 para o tiro, nos dois interruptores azuis o jogador 1 seleciona as coordenadas dos seus dois barcos e pelo interruptor vermelho(onde a alavanca 6 dá o tiro e as coordenadas são digitadas nas 1, 2, 3 e 4) o jogador seleciona o ataque.

Os leds adicionais azul e cinza ajudam na visualização das coordenadas dos dois barcos e o laranja no tiro, após serem selecionado a posição de tiro o led vermelho ou verde acenderá para informar se algum dos barcos foi acertado.

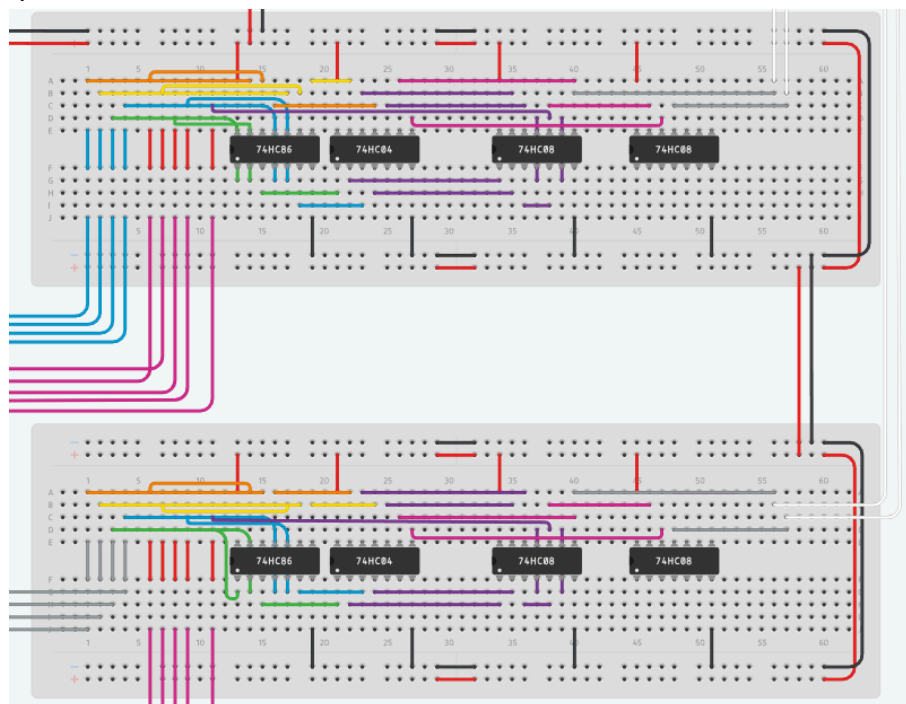
- Codificador:



O codificador está dividido por partes cada protoboard representa uma partição dele conforme visto no Logisim.

As entradas vem pelos cabos laranjas onde cada um representa 1 bit e são distribuídas para as outras protoboards respectivamente. Após o “processamento” do circuito ele retorna cada bit codificado pelos cabos roxos.

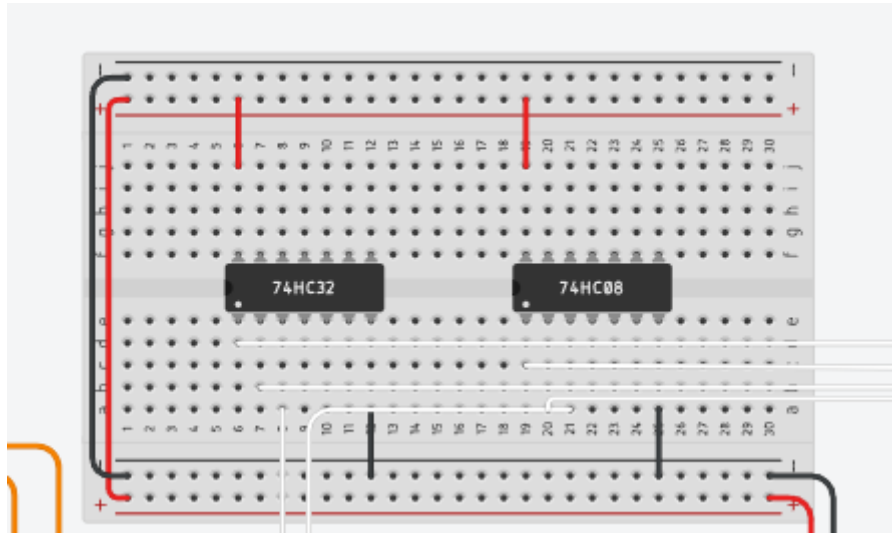
- Comparador:



Cada barco possui seu próprio codificador, mas que em estrutura são iguais:
As entradas são azul e cinza para barcos e rosa para os números do tiro já codificado.

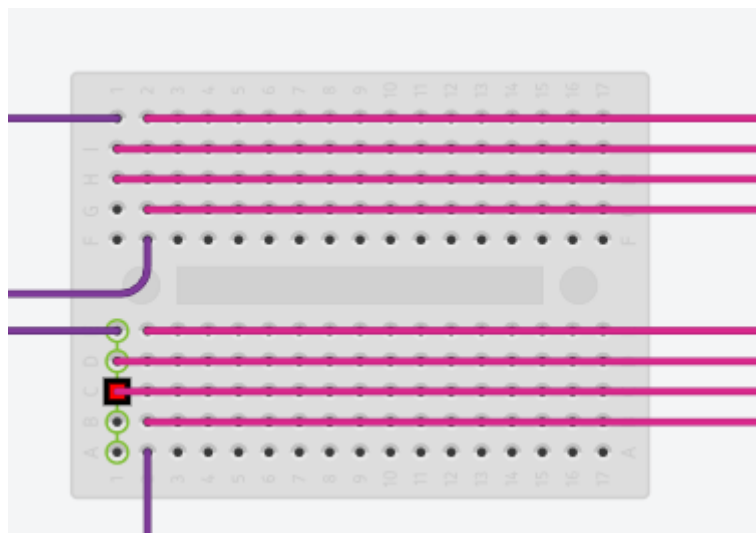
A saída é representada pelos cabos brancos.

- Leds de saída:



Com base nas saídas do codificador, elas passam por essa parte para determinar qual led irá acender, verde(acertou) ou vermelho(errou).

- Extra:



Essa parte é usada para facilitar a distribuição de cabos pelo circuito, as saídas do codificador passam por aqui e vão como entradas para o comparador.

VÍDEO para explicação:

Drive: https://drive.google.com/drive/folders/1ssHnSnFO0PQtGHFVlStNsnlMDwlBPo_?usp=sharing